

Taller Investigativo

Aprendiz: Andrés Felipe Pabón Ardila

Programa: Análisis y Desarrollo de Software

Ficha: 3407799

Instructora: Carlos Julio Cadena

Neiva

2026

Tema 1. Operador Ternario (?:)

1. Nombre del método abreviado o sintaxis comprimida

Operador Ternario (?:)

1.2. Explicación

El operador ternario es una estructura condicional abreviada que permite reemplazar un bloque simple de if-else por una sola línea de código. Evalúa una condición y devuelve un valor si la condición es verdadera y otro diferente si es falsa.

Su sintaxis es:

condición ? valorSiVerdadero : valorSiFalso;

Este operador es muy utilizado cuando únicamente se necesita asignar un valor dependiendo del resultado de una condición.

1.3. Ventajas

- Reduce la cantidad de líneas de código.
- Hace que el código sea más limpio y fácil de leer cuando las condiciones son simples.
- Facilita la asignación de valores sin utilizar un bloque completo de if-else.
- Ayuda a escribir código más compacto.

1.4. Desventajas

- Puede dificultar la lectura cuando la condición es compleja.
- No es recomendable anidar varios operadores ternarios.
- No reemplaza estructuras if-else que contienen múltiples instrucciones.

1.5. ¿Cuándo utilizarlo?

Se recomienda utilizar el operador ternario cuando únicamente se necesita decidir entre dos valores a partir de una condición sencilla, especialmente en asignaciones de variables o al retornar un valor.

1.6. Ejemplo de código

Forma tradicional

```
int edad = 20;
```

```
String mensaje;
```

```
if (edad >= 18) {  
    mensaje = "Mayor de edad";  
} else {  
    mensaje = "Menor de edad";  
}
```

```
System.out.println(mensaje);
```

Forma abreviada con operador ternario

```
int edad = 20;
```

```
String mensaje = (edad >= 18) ? "Mayor de edad" : "Menor de edad";
```

```
System.out.println(mensaje);
```

1.7. Comparación con una implementación tradicional

Implementación tradicional	Operador ternario
Utiliza la estructura if-else.	Utiliza una única expresión.
Requiere más líneas de código.	Reduce considerablemente el número de líneas.
Es adecuada para lógica compleja.	Es recomendable únicamente para condiciones simples.
Puede contener varias instrucciones dentro de cada bloque.	Solo permite devolver un valor por cada condición.

Tema 2. Operadores de Asignación Compuesta (+=, -=, *=, /=, %=)

1. Nombre del método abreviado o sintaxis comprimida

Operadores de Asignación Compuesta (+=, -=, *=, /=, %=)

2.2. Explicación

Los operadores de asignación compuesta permiten realizar una operación matemática y asignar el resultado a la misma variable en una sola instrucción. Son una forma abreviada de escribir operaciones que modifican el valor de una variable sin repetir su nombre.

Por ejemplo, en lugar de escribir:

```
numero = numero + 5;
```

se puede escribir:

```
numero += 5;
```

El resultado es el mismo, pero el código es más corto y fácil de leer.

2.3. Ventajas

- Reduce la cantidad de código escrito.
- Evita repetir el nombre de la variable.
- Hace el código más limpio y legible.
- Es muy utilizado para actualizar contadores, acumuladores y operaciones matemáticas.

2.4. Desventajas

- Puede resultar confuso para personas que están comenzando a programar.
- Si se utiliza en expresiones complejas, puede dificultar la comprensión del código.
- Solo es útil cuando la variable se modifica a sí misma.

2.5. ¿Cuándo utilizarlo?

Se recomienda utilizar los operadores de asignación compuesta cuando una variable necesita actualizar su propio valor mediante una operación matemática. Son muy comunes en ciclos, cálculos, acumuladores y contadores.

2.6. Ejemplo de código

Forma tradicional

```
int numero = 10;
```

```
numero = numero + 5;
```

```
numero = numero - 2;
```

```
numero = numero * 3;
```

```
numero = numero / 2;
```

```
numero = numero % 4;
```

```
System.out.println(numero);
```

Forma abreviada

```
int numero = 10;
```

```
numero += 5;
```

```
numero -= 2;
```

```
numero *= 3;
```

```
numero /= 2;
```

```
numero %= 4;
```

```
System.out.println(numero);
```

2.7. Comparación con una implementación tradicional

Implementación tradicional

Repite el nombre de la variable en cada operación.

Requiere más líneas y más escritura.

Operadores de asignación compuesta

Solo se escribe la variable una vez.

Reduce la cantidad de código.

Implementación tradicional

Es fácil de entender para principiantes.

Produce el mismo resultado.

Operadores de asignación compuesta

Es más práctica cuando ya se conoce la sintaxis.

Produce el mismo resultado de forma más breve.

Tema 3. Operadores de Incremento y Decremento (++ y --)

1. Nombre del método abreviado o sintaxis comprimida

Operadores de Incremento (++) y Decremento (--)

3.2. Explicación

Los operadores de incremento (++) y decremento (--) permiten aumentar o disminuir el valor de una variable en una unidad de forma abreviada. En lugar de escribir una operación completa, basta con utilizar uno de estos operadores.

- ++ incrementa el valor de la variable en 1.
- -- disminuye el valor de la variable en 1.

Estos operadores pueden utilizarse antes o después de la variable:

- **Preincremento (++variable):** primero incrementa la variable y luego utiliza el nuevo valor.
- **Postincremento (variable++):** primero utiliza el valor actual y después realiza el incremento.

Lo mismo ocurre con el operador de decremento.

3.3. Ventajas

- Reduce la cantidad de código.
- Hace más sencilla la actualización de contadores.
- Es muy utilizado en ciclos como for y while.
- Mejora la legibilidad cuando solo se necesita aumentar o disminuir una unidad.

3.4. Desventajas

- El uso de preincremento y postincremento puede generar confusión en principiantes.
- En expresiones complejas puede dificultar la comprensión del código.
- Solo permite incrementar o disminuir de uno en uno.

3.5. ¿Cuándo utilizarlo?

Se recomienda utilizar estos operadores cuando se necesita aumentar o disminuir una variable en una unidad, especialmente en contadores, recorridos de arreglos y estructuras repetitivas.

3.6. Ejemplo de código

Forma tradicional

```
int contador = 5;
```

```
contador = contador + 1;
```

```
contador = contador - 1;
```

```
System.out.println(contador);
```

Forma abreviada

```
int contador = 5;
```

```
contador++;
```

```
contador--;
```

```
System.out.println(contador);
```

Ejemplo de preincremento y postincremento

```
int numero = 10;
```

```
System.out.println(numero++); // Imprime 10
```

```
System.out.println(numero); // Imprime 11
```

```
System.out.println(++numero); // Imprime 12
```

3.7. Comparación con una implementación tradicional

Implementación tradicional	Operadores ++ y --
Se escribe <code>variable = variable + 1</code> o <code>variable = variable - 1</code> .	Solo se utiliza <code>variable++</code> o <code>variable--</code> .
Requiere más líneas de código.	Reduce la cantidad de código.
Es fácil de entender para principiantes.	Es más práctico y utilizado por programadores con experiencia.
Permite incrementar o disminuir cualquier cantidad.	Solo incrementa o disminuye una unidad por operación.

Tema 4. Ciclo for-each

1. Nombre del método abreviado o sintaxis comprimida

Ciclo for-each

4.2. Explicación

El ciclo for-each es una versión simplificada del ciclo for tradicional que permite recorrer todos los elementos de un arreglo o una colección sin necesidad de utilizar un índice. Su principal objetivo es hacer el código más limpio, legible y fácil de mantener.

La sintaxis general es:

```
for (TipoDato elemento : coleccion) {  
    // Código a ejecutar  
}
```

En cada iteración, la variable `elemento` toma el valor del siguiente elemento del arreglo o colección hasta recorrerlos todos.

4.3. Ventajas

- Reduce la cantidad de código.
- No es necesario controlar un índice.

- Disminuye la posibilidad de errores al recorrer arreglos o colecciones.
- Hace el código más legible y fácil de comprender.

4.4. Desventajas

- No permite conocer directamente la posición (índice) de cada elemento.
- No es recomendable cuando se necesita modificar los elementos mientras se recorren.
- Solo puede utilizarse para recorrer arreglos o colecciones completas.

4.5. ¿Cuándo utilizarlo?

Se recomienda utilizar el ciclo for-each cuando únicamente se necesita leer o mostrar todos los elementos de un arreglo o colección, sin importar su posición.

4.6. Ejemplo de código

Forma tradicional

```
String[] nombres = {"Ana", "Luis", "Carlos", "María"};
```

```
for (int i = 0; i < nombres.length; i++) {
    System.out.println(nombres[i]);
}
```

Forma abreviada con for-each

```
String[] nombres = {"Ana", "Luis", "Carlos", "María"};
```

```
for (String nombre : nombres) {
    System.out.println(nombre);
}
```

4.7. Comparación con una implementación tradicional

Implementación tradicional

Utiliza un índice para recorrer los elementos.

Ciclo for-each

Recorre los elementos directamente.

Implementación tradicional

Requiere más código.

Permite acceder a la posición de cada elemento.

Es útil cuando se necesita modificar posiciones específicas.

Ciclo for-each

Es más corto y fácil de leer.

No permite acceder directamente al índice.

Es ideal cuando solo se desea recorrer y leer los datos.

Tema 5. Inferencia de Tipos con var

1. Nombre del método abreviado o sintaxis comprimida

Inferencia de Tipos con var

5.1 Explicación

La inferencia de tipos con var es una característica incorporada en Java desde la versión 10 que permite declarar variables locales sin escribir explícitamente su tipo de dato. El compilador identifica automáticamente el tipo según el valor que se asigna a la variable en el momento de su declaración.

Esto ayuda a reducir la cantidad de código escrito y mejora la legibilidad, especialmente cuando se trabajan con tipos de datos largos o complejos.

Es importante tener en cuenta que var **no significa que la variable pueda cambiar de tipo**. Una vez el compilador determina el tipo de dato, este permanece fijo durante toda la ejecución del programa.

5.2 Ventajas

- Reduce la cantidad de código.
- Hace que las declaraciones de variables sean más limpias.
- Facilita el trabajo con objetos de tipos complejos o genéricos.
- Mejora la legibilidad cuando el tipo de dato es evidente.

5.3 Desventajas

- Puede dificultar la comprensión del código si no es claro qué tipo de dato contiene la variable.
- Solo puede utilizarse en variables locales.

- No puede usarse para declarar atributos de una clase, parámetros de métodos o valores null sin contexto.

5.4 ¿Cuándo utilizarlo?

Se recomienda utilizar var cuando el tipo de dato sea fácil de identificar a partir del valor asignado. Es especialmente útil al trabajar con colecciones, objetos y expresiones largas donde escribir el tipo completo resulta innecesario.

5.5 Ejemplo de código

Forma tradicional

```
String nombre = "Andrés";
```

```
int edad = 20;
```

```
double promedio = 4.5;
```

Forma abreviada con var

```
var nombre = "Andrés";
```

```
var edad = 20;
```

```
var promedio = 4.5;
```

```
System.out.println(nombre);
```

```
System.out.println(edad);
```

```
System.out.println(promedio);
```

5.6 Comparación con una implementación tradicional

Implementación tradicional

Se escribe el tipo de dato explícitamente.

Requiere más escritura.

El tipo es visible desde el inicio.

Funciona en cualquier declaración de variable.

Inferencia de tipos con var

El compilador deduce el tipo automáticamente.

Reduce la cantidad de código.

El tipo debe deducirse a partir del valor asignado.

Solo puede utilizarse en variables locales.

Tema 6. List.of()

1. Nombre del método abreviado o sintaxis comprimida

Método List.of()

6.1 Explicación

El método List.of() fue incorporado en Java 9 y permite crear listas de forma rápida y sencilla en una sola línea de código. Antes de su aparición, era necesario crear una lista y agregar cada elemento utilizando el método add().

Las listas creadas con List.of() son **inmutables**, lo que significa que una vez creadas no es posible agregar, eliminar o modificar sus elementos.

Este método simplifica la creación de listas cuando los datos no cambiarán durante la ejecución del programa.

6.2 Ventajas

- Reduce la cantidad de código.
- Permite crear listas en una sola instrucción.
- Hace el código más limpio y legible.
- Evita modificaciones accidentales gracias a que la lista es inmutable.

6.3 Desventajas

- No permite agregar, eliminar ni modificar elementos después de crear la lista.
- Solo está disponible a partir de Java 9.
- Si se intenta modificar la lista, se genera una excepción (UnsupportedOperationException).

6.4 ¿Cuándo utilizarlo?

Se recomienda utilizar List.of() cuando se necesita una lista con valores fijos que no cambiarán durante la ejecución del programa, como listas de países, meses del año, colores o configuraciones.

6.5 Ejemplo de código

Forma tradicional

```
import java.util.ArrayList;  
  
import java.util.List;
```

```
List<String> frutas = new ArrayList<>();
```

```
frutas.add("Manzana");
```

```
frutas.add("Banano");
```

```
frutas.add("Naranja");
```

```
System.out.println(frutas);
```

Forma abreviada con List.of()

```
import java.util.List;
```

```
List<String> frutas = List.of("Manzana", "Banano", "Naranja");
```

```
System.out.println(frutas);
```

6.6 Comparación con una implementación tradicional

Implementación tradicional	List.of()
Se crea la lista y luego se agregan los elementos con add().	Los elementos se agregan directamente al crear la lista.
Requiere más líneas de código.	Reduce considerablemente el código.
La lista puede modificarse.	La lista es inmutable.
Disponible en todas las versiones modernas de Java.	Disponible desde Java 9.

Tema 7. Map.of()

1. Nombre del método abreviado o sintaxis comprimida

Método Map.of()

7.1 Explicación

El método `Map.of()` fue introducido en Java 9 y permite crear mapas de forma rápida y sencilla en una sola línea de código. Un **Map** es una estructura de datos que almacena información en pares de **clave** y **valor**, donde cada clave es única.

Antes de la aparición de `Map.of()`, era necesario crear el mapa y agregar cada elemento utilizando el método `put()`. Con esta nueva sintaxis, los pares clave-valor se definen directamente al momento de crear el mapa.

Los mapas creados con `Map.of()` son **inmutables**, por lo que no es posible agregar, eliminar o modificar sus elementos después de su creación.

7.2 Ventajas

- Reduce la cantidad de código.
- Permite crear mapas en una sola instrucción.
- Hace el código más limpio y fácil de leer.
- Evita modificaciones accidentales gracias a que el mapa es inmutable.

7.3 Desventajas

- No permite agregar ni eliminar elementos después de su creación.
- No admite claves duplicadas.
- Solo está disponible a partir de Java 9.

7.4 ¿Cuándo utilizarlo?

Se recomienda utilizar `Map.of()` cuando se necesita almacenar una colección de datos fijos organizados por clave y valor, como códigos, configuraciones, abreviaturas o información que no cambiará durante la ejecución del programa.

7.5 Ejemplo de código

Forma tradicional

```
import java.util.HashMap;  
import java.util.Map;
```

```
Map<Integer, String> estudiantes = new HashMap<>();
```

```
estudiantes.put(1, "Andrés");
```

```
estudiantes.put(2, "Laura");  
estudiantes.put(3, "Carlos");
```

```
System.out.println(estudiantes);
```

Forma abreviada con Map.of()

```
import java.util.Map;
```

```
Map<Integer, String> estudiantes = Map.of(  
    1, "Andrés",  
    2, "Laura",  
    3, "Carlos"  
);
```

```
System.out.println(estudiantes);
```

7.6 Comparación con una implementación tradicional

Implementación tradicional	Map.of()
Se crea el mapa y luego se agregan los elementos con put().	Las claves y valores se agregan directamente al crear el mapa.
Requiere más líneas de código.	Reduce la cantidad de código.
El mapa puede modificarse.	El mapa es inmutable.
Disponible en versiones anteriores de Java.	Disponible desde Java 9.

Tema 8. Método forEach()

1. Nombre del método abreviado o sintaxis comprimida

Método forEach()

8.1 Explicación

El método `forEach()` fue incorporado en Java 8 y permite recorrer todos los elementos de una colección de una forma más sencilla y moderna. A diferencia del ciclo `for` o `for-each`, `forEach()` es un método propio de las colecciones y recibe como parámetro una acción que se ejecutará sobre cada elemento.

Generalmente se utiliza junto con expresiones Lambda o referencias a métodos, lo que hace que el código sea más corto y fácil de leer.

8.2 Ventajas

- Reduce la cantidad de código.
- Hace el recorrido de colecciones más sencillo.
- Mejora la legibilidad del programa.
- Se integra fácilmente con las expresiones Lambda y la API Streams.

8.3 Desventajas

- No permite acceder directamente al índice de los elementos.
- Puede resultar difícil de comprender para quienes comienzan a aprender Java.
- No es la mejor opción cuando se necesita modificar la colección durante el recorrido.

8.4 ¿Cuándo utilizarlo?

Se recomienda utilizar `forEach()` cuando se necesita recorrer todos los elementos de una colección para realizar una misma acción, como mostrar información, realizar cálculos o procesar datos.

8.5 Ejemplo de código

Forma tradicional

```
import java.util.List;
```

```
public class Ejemplo {
```

```
    public static void main(String[] args) {
```

```
        List<String> ciudades = List.of("Bogotá", "Medellín", "Neiva");
```

```

        for (String ciudad : ciudades) {
            System.out.println(ciudad);
        }
    }
}

```

Forma abreviada con `forEach()`

```

import java.util.List;

public class Ejemplo {

    public static void main(String[] args) {

        List<String> ciudades = List.of("Bogotá", "Medellín", "Neiva");

        ciudades.forEach(ciudad -> System.out.println(ciudad));

    }
}

```

8.6 Comparación con una implementación tradicional

Implementación tradicional	Método <code>forEach()</code>
Utiliza un ciclo <code>for</code> o <code>for-each</code> para recorrer la colección.	Utiliza el método <code>forEach()</code> propio de la colección.
Requiere escribir la estructura del ciclo.	Reduce la cantidad de código.
Es fácil de entender para principiantes.	Es más moderno y suele combinarse con expresiones Lambda.
Permite mayor control sobre el recorrido.	Es ideal cuando solo se necesita ejecutar una acción sobre cada elemento.

Tema 9. Operador Diamond (<>)

1. Nombre del método abreviado o sintaxis comprimida

Operador Diamond (<>)

9.1 Explicación

El operador Diamond (<>) fue incorporado en Java 7 para simplificar la creación de objetos genéricos. Su función es evitar repetir los tipos de datos al momento de instanciar una colección u otro objeto genérico, ya que el compilador los deduce automáticamente.

Antes de su incorporación era necesario escribir los tipos de datos dos veces: una al declarar la variable y otra al crear el objeto. Con el operador Diamond solo es necesario escribirlos una vez, haciendo el código más limpio y fácil de mantener.

9.2 Ventajas

- Reduce la cantidad de código.
- Evita repetir los tipos de datos.
- Hace el código más limpio y legible.
- Disminuye la posibilidad de cometer errores al escribir los tipos genéricos.

9.3 Desventajas

- Solo puede utilizarse con clases genéricas.
- Requiere que el compilador pueda inferir correctamente los tipos.
- En algunos casos complejos puede ser necesario especificar los tipos manualmente.

9.4 ¿Cuándo utilizarlo?

Se recomienda utilizar el operador Diamond siempre que se creen objetos genéricos como ArrayList, HashMap, HashSet u otras colecciones, ya que simplifica la declaración del código sin cambiar su funcionamiento.

9.5 Ejemplo de código

Forma tradicional

```
import java.util.ArrayList;
```

```
ArrayList<String> nombres = new ArrayList<String>();
```

```
nombres.add("Andrés");
```

```
nombres.add("Laura");
```

```
nombres.add("Carlos");
```

```
System.out.println(nombres);
```

Forma abreviada con el operador Diamond

```
import java.util.ArrayList;
```

```
ArrayList<String> nombres = new ArrayList<>();
```

```
nombres.add("Andrés");
```

```
nombres.add("Laura");
```

```
nombres.add("Carlos");
```

```
System.out.println(nombres);
```

9.6 Comparación con una implementación tradicional

Implementación tradicional

Es necesario escribir el tipo genérico dos veces.

Requiere más código.

Es más repetitivo.

Produce el mismo resultado.

Operador Diamond (<>)

El tipo se escribe una sola vez.

Reduce la cantidad de código.

Hace el código más limpio y fácil de leer.

Produce el mismo resultado de forma más sencilla.

Tema 10. String.format()

1. Nombre del método abreviado o sintaxis comprimida

Método String.format()

10.1 Explicación

El método `String.format()` permite crear cadenas de texto con un formato específico sin necesidad de concatenar varias variables. En lugar de unir textos utilizando el operador `+`, este método emplea marcadores de formato para insertar valores dentro de una cadena.

Algunos de los marcadores más utilizados son:

- `%s` → Cadenas de texto (String).
- `%d` → Números enteros (int).
- `%f` → Números decimales (float o double).
- `%c` → Caracteres (char).

Esta forma de escribir el código facilita la lectura y organización de los mensajes que se muestran al usuario.

10.2 Ventajas

- Hace el código más limpio y ordenado.
- Evita concatenaciones largas con el operador `+`.
- Facilita la creación de mensajes con diferentes tipos de datos.
- Permite controlar el formato de los números y los textos.

10.3 Desventajas

- Es necesario conocer los especificadores de formato (`%s`, `%d`, `%f`, entre otros).
- Puede resultar innecesario para mensajes muy simples.
- Un especificador incorrecto puede generar errores en tiempo de ejecución.

10.4 ¿Cuándo utilizarlo?

Se recomienda utilizar `String.format()` cuando se necesita construir mensajes que combinan texto con variables, especialmente en reportes, resultados de cálculos o información que será mostrada al usuario.

10.5 Ejemplo de código

Forma tradicional

```
String nombre = "Andrés";
```

```
int edad = 20;
```

```
String mensaje = "Mi nombre es " + nombre + " y tengo " + edad + " años.";
```

```
System.out.println(mensaje);
```

Forma abreviada con `String.format()`

```
String nombre = "Andrés";
```

```
int edad = 20;
```

```
String mensaje = String.format("Mi nombre es %s y tengo %d años.", nombre, edad);
```

```
System.out.println(mensaje);
```

10.6 Comparación con una implementación tradicional

Implementación tradicional	<code>String.format()</code>
Utiliza el operador + para unir cadenas.	Utiliza marcadores de formato para insertar valores.
Puede ser difícil de leer cuando hay muchas variables.	Hace el código más organizado y legible.
Es adecuada para mensajes sencillos.	Es ideal para construir mensajes más complejos y ordenados.
Requiere varias concatenaciones.	Reduce la cantidad de concatenaciones necesarias.

Tema 11. Método removelf()

1. Nombre del método abreviado o sintaxis comprimida

Método removelf()

11.1 Explicación

El método removelf() fue introducido en Java 8 y permite eliminar de una colección todos los elementos que cumplan una condición específica. Antes de este método, era necesario recorrer la colección utilizando un ciclo y eliminar los elementos manualmente.

removelf() recibe una condición (predicado) y elimina automáticamente todos los elementos que la satisfacen, haciendo el código más corto y fácil de entender.

11.2 Ventajas

- Reduce la cantidad de código.
- Facilita la eliminación de elementos según una condición.
- Hace el código más limpio y legible.
- Evita recorrer la colección manualmente para eliminar elementos.

11.3 Desventajas

- Solo puede utilizarse en colecciones que permitan modificaciones.
- Requiere comprender el uso de condiciones o expresiones Lambda.
- No está disponible en versiones anteriores a Java 8.

11.4 ¿Cuándo utilizarlo?

Se recomienda utilizar removelf() cuando se necesite eliminar uno o varios elementos de una colección que cumplan una condición determinada, como borrar números negativos, eliminar nombres vacíos o filtrar datos innecesarios.

11.5 Ejemplo de código

Forma tradicional

```
import java.util.ArrayList;  
  
import java.util.List;
```

```
List<Integer> numeros = new ArrayList<>();
```

```
numeros.add(10);
```

```
numeros.add(25);
```

```
numeros.add(8);
```

```
numeros.add(30);
```

```
for (int i = numeros.size() - 1; i >= 0; i--) {
```

```
    if (numeros.get(i) > 20) {
```

```
        numeros.remove(i);
```

```
    }
```

```
}
```

```
System.out.println(numeros);
```

Forma abreviada con removeIf()

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
List<Integer> numeros = new ArrayList<>();
```

```
numeros.add(10);
```

```
numeros.add(25);
```

```
numeros.add(8);
```

```
numeros.add(30);
```

```
numeros.removeIf(numero -> numero > 20);
```

```
System.out.println(numeros);
```

11.6 Comparación con una implementación tradicional

Implementación tradicional	Método <code>removeIf()</code>
Es necesario recorrer la colección con un ciclo.	Elimina los elementos directamente mediante una condición.
Requiere más líneas de código.	Reduce considerablemente el código.
Puede ser más complejo de implementar.	Es más sencillo y fácil de leer.
El programador controla el recorrido de la colección.	El método realiza automáticamente el proceso de eliminación.

Tema 12. `Set.of()`

1. Nombre del método abreviado o sintaxis comprimida

Método `Set.of()`

12.1 Explicación

El método `Set.of()` fue introducido en Java 9 y permite crear conjuntos (`Set`) de forma rápida y sencilla en una sola línea de código. Un **Set** es una colección que almacena elementos únicos, es decir, no permite valores duplicados.

Antes de la aparición de `Set.of()`, era necesario crear un conjunto y agregar cada elemento utilizando el método `add()`. Con este método moderno, los elementos se especifican directamente al momento de crear el conjunto.

Los conjuntos creados con `Set.of()` son **inmutables**, por lo que no es posible agregar, eliminar o modificar sus elementos después de su creación.

12.2 Ventajas

- Reduce la cantidad de código.
- Permite crear conjuntos en una sola instrucción.
- Garantiza que no existan elementos duplicados.
- Hace el código más limpio y fácil de leer.

12.3 Desventajas

- No permite modificar el conjunto después de crearlo.
- Si se incluyen elementos duplicados, se genera una excepción.
- Solo está disponible a partir de Java 9.

12.4 ¿Cuándo utilizarlo?

Se recomienda utilizar `Set.of()` cuando se necesita un conjunto de datos fijos sin elementos repetidos, como códigos, categorías, colores o días de la semana.

12.5 Ejemplo de código

Forma tradicional

```
import java.util.HashSet;
```

```
import java.util.Set;
```

```
Set<String> colores = new HashSet<>();
```

```
colores.add("Rojo");
```

```
colores.add("Azul");
```

```
colores.add("Verde");
```

```
System.out.println(colores);
```

Forma abreviada con `Set.of()`

```
import java.util.Set;
```

```
Set<String> colores = Set.of("Rojo", "Azul", "Verde");
```

```
System.out.println(colores);
```

12.6 Comparación con una implementación tradicional

Implementación tradicional

Se crea el conjunto y luego se agregan los elementos con add().

Requiere más líneas de código.

El conjunto puede modificarse.

Disponible en versiones anteriores de Java.

Set.of()

Los elementos se agregan directamente al crear el conjunto.

Reduce la cantidad de código.

El conjunto es inmutable.

Disponible desde Java 9.

Tema 13. Clase Optional

1. Nombre del método abreviado o sintaxis comprimida

Clase Optional

13.1 Explicación

La clase Optional fue introducida en Java 8 para representar un valor que puede estar presente o ausente. Su principal objetivo es evitar errores de tipo NullPointerException, uno de los problemas más comunes en Java cuando se intenta utilizar una variable que contiene null.

En lugar de devolver directamente un objeto que podría ser null, un método puede devolver un Optional, obligando al programador a verificar si el valor existe antes de utilizarlo.

13.2 Ventajas

- Reduce la posibilidad de errores por valores null.
- Hace el código más seguro y fácil de entender.
- Obliga a verificar la existencia de un valor antes de utilizarlo.
- Mejora la calidad y el mantenimiento del código.

13.3 Desventajas

- Puede hacer el código un poco más extenso en casos sencillos.
- Requiere conocer los métodos propios de la clase Optional.
- No es recomendable utilizarlo como atributo de una clase.

13.4 ¿Cuándo utilizarlo?

Se recomienda utilizar Optional cuando un método puede o no devolver un resultado, evitando retornar null y disminuyendo el riesgo de errores durante la ejecución del programa.

13.5 Ejemplo de código

Forma tradicional

```
String nombre = null;
```

```
if (nombre != null) {  
    System.out.println(nombre.toUpperCase());  
} else {  
    System.out.println("No existe un nombre.");  
}
```

Forma abreviada con Optional

```
import java.util.Optional;
```

```
Optional<String> nombre = Optional.ofNullable(null);
```

```
System.out.println(nombre.orElse("No existe un nombre."));
```

13.6 Comparación con una implementación tradicional

Implementación tradicional	Clase Optional
Se trabaja directamente con valores null.	Encapsula el valor dentro de un objeto Optional.
Es necesario verificar manualmente si el valor es null.	Proporciona métodos para manejar la ausencia de valores de forma segura.
Existe mayor riesgo de un NullPointerException.	Reduce considerablemente ese riesgo.

Implementación tradicional

Es la forma clásica de trabajar con valores opcionales.

Clase Optional

Es una alternativa más moderna y segura introducida en Java 8.

Tema 14. Switch Expression

1. Nombre del método abreviado o sintaxis comprimida

Switch Expression

14.1 Explicación

La **Switch Expression** es una mejora introducida en Java 14 que permite escribir estructuras switch de una forma más corta, clara y segura. A diferencia del switch tradicional, utiliza el operador `->` para indicar la acción que debe ejecutarse en cada caso y puede devolver un valor directamente, evitando el uso repetitivo de `break`.

Esta característica hace que el código sea más fácil de leer y reduce la posibilidad de errores ocasionados por olvidar la instrucción `break`.

14.2 Ventajas

- Reduce la cantidad de código.
- No es necesario utilizar la instrucción `break`.
- Hace el código más limpio y fácil de leer.
- Permite devolver un valor directamente desde el switch.

14.3 Desventajas

- Solo está disponible en versiones recientes de Java.
- Puede resultar desconocida para quienes solo han utilizado el switch tradicional.
- No es compatible con versiones antiguas del lenguaje.

14.4 ¿Cuándo utilizarlo?

Se recomienda utilizar **Switch Expression** cuando se necesita seleccionar una opción entre varios casos y asignar un valor de forma rápida y sencilla, haciendo el código más organizado y fácil de mantener.

14.5 Ejemplo de código

Forma tradicional

```
int dia = 3;
```

```
String nombreDia;
```

```
switch (dia) {  
    case 1:  
        nombreDia = "Lunes";  
        break;  
    case 2:  
        nombreDia = "Martes";  
        break;  
    case 3:  
        nombreDia = "Miércoles";  
        break;  
    default:  
        nombreDia = "Día no válido";  
}
```

```
System.out.println(nombreDia);
```

Forma abreviada con Switch Expression

```
int dia = 3;
```

```
String nombreDia = switch (dia) {  
    case 1 -> "Lunes";  
    case 2 -> "Martes";  
    case 3 -> "Miércoles";
```

```
default -> "Día no válido";  
};
```

```
System.out.println(nombreDia);
```

14.6 Comparación con una implementación tradicional

Implementación tradicional	Switch Expression
Utiliza case junto con break.	Utiliza case con el operador ->.
Requiere más líneas de código.	Reduce la cantidad de código.
Es posible olvidar un break y provocar errores.	Elimina ese riesgo al no necesitar break.
Se utiliza principalmente para ejecutar acciones.	También puede devolver un valor directamente.

Tema 15. Referencias a Métodos (::)

1. Nombre del método abreviado o sintaxis comprimida

Referencias a Métodos (::)

15.1 Explicación

Las referencias a métodos (::) fueron introducidas en Java 8 como una forma más abreviada de utilizar expresiones Lambda cuando la operación consiste únicamente en llamar a un método ya existente.

En lugar de escribir una expresión Lambda completa, se puede utilizar el operador :: para indicar qué método debe ejecutarse. Esto hace que el código sea más corto, claro y fácil de leer.

Las referencias a métodos se utilizan con frecuencia junto con colecciones, `forEach()` y la API Streams.

15.2 Ventajas

- Reduce la cantidad de código.
- Hace el programa más limpio y legible.
- Evita escribir expresiones Lambda innecesarias.

- Facilita el mantenimiento del código.

15.3 Desventajas

- Solo puede utilizarse cuando existe un método compatible con la operación requerida.
- Puede ser difícil de comprender para quienes están comenzando a aprender Java.
- No reemplaza a las expresiones Lambda en todos los casos.

15.4 ¿Cuándo utilizarlo?

Se recomienda utilizar referencias a métodos cuando una expresión Lambda únicamente llama a un método existente, ya que permiten escribir el mismo código de una forma más sencilla y elegante.

15.5 Ejemplo de código

Forma tradicional con Expresión Lambda

```
import java.util.List;

public class Ejemplo {
    public static void main(String[] args) {

        List<String> nombres = List.of("Andrés", "Laura", "Carlos");

        nombres.forEach(nombre -> System.out.println(nombre));
    }
}
```

Forma abreviada con Referencia a Métodos

```
import java.util.List;

public class Ejemplo {
    public static void main(String[] args) {
```

```

List<String> nombres = List.of("Andrés", "Laura", "Carlos");

nombres.forEach(System.out::println);
}
}

```

15.6 Comparación con una implementación tradicional

Implementación tradicional	Referencias a Métodos (::)
Utiliza una expresión Lambda para llamar a un método.	Utiliza el operador :: para referenciar directamente el método.
Requiere escribir más código.	Reduce la cantidad de código.
Es adecuada para cualquier operación.	Es ideal cuando solo se llama a un método existente.
Puede ser más extensa.	Hace el código más limpio y fácil de leer.

Tema 16. String.join()

1. Nombre del método abreviado o sintaxis comprimida

Método String.join()

16.1 Explicación

El método `String.join()` fue introducido en Java 8 y permite unir varias cadenas de texto utilizando un separador específico, como una coma, un espacio o un guion. Antes de este método, era común utilizar el operador `+` o ciclos para concatenar varias cadenas.

Con `String.join()`, el código es más limpio, fácil de leer y evita realizar múltiples concatenaciones.

La sintaxis general es:

```
String.join(separador, elementos);
```

16.2 Ventajas

- Reduce la cantidad de código.
- Facilita la unión de varias cadenas de texto.
- Hace el código más limpio y legible.
- Permite definir fácilmente el separador entre los elementos.

16.3 Desventajas

- Solo es útil cuando se necesita unir cadenas de texto.
- No reemplaza otros métodos de manipulación de cadenas.
- Requiere que los elementos sean de tipo String.

16.4 ¿Cuándo utilizarlo?

Se recomienda utilizar `String.join()` cuando se necesita unir varios textos utilizando un mismo separador, por ejemplo, al crear listas de nombres, direcciones, mensajes o reportes.

16.5 Ejemplo de código

Forma tradicional

```
String ciudades = "Neiva" + ", " + "Bogotá" + ", " + "Medellín";
```

```
System.out.println(ciudades);
```

Forma abreviada con `String.join()`

```
String ciudades = String.join(", ", "Neiva", "Bogotá", "Medellín");
```

```
System.out.println(ciudades);
```

16.6 Comparación con una implementación tradicional

Implementación tradicional	<code>String.join()</code>
Utiliza el operador <code>+</code> para unir cadenas.	Une las cadenas mediante un separador definido.
Requiere varias concatenaciones.	Reduce la cantidad de código.

Implementación tradicional

Puede ser difícil de leer cuando hay muchos elementos.

Es adecuada para pocas cadenas.

String.join()

Hace el código más claro y organizado.

Es ideal para unir varias cadenas con un mismo formato.

Tema 17. String.repeat()

1. Nombre del método abreviado o sintaxis comprimida

Método String.repeat()

17.1 Explicación

El método `String.repeat()` fue incorporado en Java 11 y permite repetir una cadena de texto un número determinado de veces sin necesidad de utilizar ciclos o concatenaciones.

Antes de este método, era necesario utilizar un `for` o `while` para repetir una cadena varias veces. Con `String.repeat()`, la tarea se realiza mediante una sola instrucción, haciendo el código más sencillo y legible.

La sintaxis general es:

```
cadena.repeat(numeroDeRepeticiones);
```

17.2 Ventajas

- Reduce la cantidad de código.
- Evita el uso de ciclos para repetir cadenas.
- Hace el código más limpio y fácil de leer.
- Es útil para generar separadores, formatos o patrones repetitivos.

17.3 Desventajas

- Solo puede utilizarse para repetir cadenas de texto.
- Está disponible únicamente a partir de Java 11.
- Si se utiliza un número muy grande de repeticiones, puede aumentar el consumo de memoria.

17.4 ¿Cuándo utilizarlo?

Se recomienda utilizar `String.repeat()` cuando se necesita repetir un texto varias veces, por ejemplo, para crear líneas divisorias, generar patrones de caracteres o construir mensajes con un formato específico.

17.5 Ejemplo de código

Forma tradicional

```
String linea = "";
```

```
for (int i = 0; i < 10; i++) {  
    linea += "-";  
}
```

```
System.out.println(linea);
```

Forma abreviada con `String.repeat()`

```
String linea = "-".repeat(10);
```

```
System.out.println(linea);
```

17.6 Comparación con una implementación tradicional

Implementación tradicional	<code>String.repeat()</code>
Utiliza un ciclo para repetir la cadena.	Repite la cadena con una sola instrucción.
Requiere más líneas de código.	Reduce considerablemente el código.
Puede ser más difícil de leer.	Hace el código más claro y sencillo.
Funciona en cualquier versión de Java. Disponible desde Java 11.	

Tema 18. Método `Arrays.toString()`

1. Nombre del método abreviado o sintaxis comprimida

Método Arrays.toString()

18.1 Explicación

El método Arrays.toString() pertenece a la clase Arrays y permite convertir un arreglo en una cadena de texto para mostrar fácilmente todos sus elementos.

Antes de utilizar este método, era necesario recorrer el arreglo con un ciclo para imprimir cada elemento. Con Arrays.toString(), basta una sola línea de código para visualizar todo el contenido del arreglo.

Este método es muy útil para depurar programas y verificar rápidamente la información almacenada en un arreglo.

18.2 Ventajas

- Reduce la cantidad de código.
- Permite mostrar un arreglo completo con una sola instrucción.
- Hace el código más limpio y fácil de leer.
- Es muy útil para depurar y comprobar el contenido de un arreglo.

18.3 Desventajas

- Solo sirve para mostrar el contenido del arreglo.
- No permite modificar los elementos del arreglo.
- Requiere importar la clase java.util.Arrays.

18.4 ¿Cuándo utilizarlo?

Se recomienda utilizar Arrays.toString() cuando se desea mostrar el contenido completo de un arreglo de forma rápida y sencilla, sin necesidad de recorrerlo con un ciclo.

18.5 Ejemplo de código

Forma tradicional

```
int[] numeros = {10, 20, 30, 40};
```

```
for (int i = 0; i < numeros.length; i++) {  
    System.out.print(numeros[i] + " ");  
}
```

Forma abreviada con `Arrays.toString()`

```
import java.util.Arrays;
```

```
int[] numeros = {10, 20, 30, 40};
```

```
System.out.println(Arrays.toString(numeros));
```

Salida:

```
[10, 20, 30, 40]
```

18.6 Comparación con una implementación tradicional

Implementación tradicional	<code>Arrays.toString()</code>
Es necesario recorrer el arreglo con un ciclo.	Muestra todo el arreglo con una sola instrucción.
Requiere más líneas de código.	Reduce la cantidad de código.
Es más lento de escribir.	Es más rápido y fácil de utilizar.
Permite personalizar la impresión.	Muestra el arreglo con un formato estándar.

Tema 19. Método `Arrays.sort()`

1. Nombre del método abreviado o sintaxis comprimida

Método `Arrays.sort()`

19.1 Explicación

El método `Arrays.sort()` pertenece a la clase `Arrays` y se utiliza para ordenar los elementos de un arreglo de forma automática. Antes de utilizar este método, era necesario implementar algoritmos de ordenamiento, como el método burbuja o selección, lo que implicaba escribir muchas más líneas de código.

Con `Arrays.sort()`, Java realiza el ordenamiento de manera rápida y eficiente mediante una sola instrucción.

19.2 Ventajas

- Reduce la cantidad de código.
- Permite ordenar arreglos de forma rápida y sencilla.
- Utiliza algoritmos de ordenamiento optimizados.
- Hace el código más limpio y fácil de mantener.

19.3 Desventajas

- Solo ordena los elementos en orden natural, a menos que se utilicen comparadores.
- Requiere importar la clase `java.util.Arrays`.
- Modifica el arreglo original.

19.4 ¿Cuándo utilizarlo?

Se recomienda utilizar `Arrays.sort()` cuando se necesita ordenar un arreglo de números, caracteres o cadenas de texto sin implementar un algoritmo de ordenamiento manualmente.

19.5 Ejemplo de código

Forma tradicional

```
int[] numeros = {8, 3, 10, 1, 5};
```

```
// Ordenamiento manual (método burbuja)
```

```
for (int i = 0; i < numeros.length - 1; i++) {  
    for (int j = 0; j < numeros.length - i - 1; j++) {  
        if (numeros[j] > numeros[j + 1]) {  
            int aux = numeros[j];  
            numeros[j] = numeros[j + 1];  
            numeros[j + 1] = aux;  
        }  
    }  
}
```

```
for (int numero : numeros) {
    System.out.print(numero + " ");
}
```

Forma abreviada con Arrays.sort()

```
import java.util.Arrays;
```

```
int[] numeros = {8, 3, 10, 1, 5};
```

```
Arrays.sort(numeros);
```

```
System.out.println(Arrays.toString(numeros));
```

Salida:

```
[1, 3, 5, 8, 10]
```

19.6 Comparación con una implementación tradicional

Implementación tradicional	Arrays.sort()
Es necesario implementar un algoritmo de ordenamiento.	Ordena el arreglo con una sola instrucción.
Requiere muchas líneas de código.	Reduce considerablemente el código.
Puede ser más difícil de comprender.	Es más sencillo y fácil de utilizar.
Depende de la implementación realizada por el programador.	Utiliza algoritmos optimizados incorporados en Java.

Tema 20. Método Math.max() y Math.min()

1. Nombre del método abreviado o sintaxis comprimida

Métodos Math.max() y Math.min()

20.1 Explicación

Los métodos `Math.max()` y `Math.min()` pertenecen a la clase `Math` y permiten obtener el mayor o el menor de dos valores mediante una sola instrucción.

Antes de utilizar estos métodos, era común emplear estructuras `if-else` para comparar dos números. Con `Math.max()` y `Math.min()`, Java realiza la comparación automáticamente, haciendo el código más corto y fácil de leer.

- `Math.max(a, b)` devuelve el **valor mayor**.
- `Math.min(a, b)` devuelve el **valor menor**.

20.2 Ventajas

- Reduce la cantidad de código.
- Evita utilizar estructuras `if-else` para comparar dos valores.
- Hace el código más limpio y fácil de entender.
- Es fácil de utilizar en operaciones matemáticas.

20.3 Desventajas

- Solo compara dos valores por llamada.
- No reemplaza algoritmos de búsqueda cuando se comparan muchos elementos.
- Solo realiza comparaciones, no modifica los datos.

20.4 ¿Cuándo utilizarlo?

Se recomienda utilizar `Math.max()` y `Math.min()` cuando se necesita conocer rápidamente cuál es el mayor o el menor entre dos números, por ejemplo, al comparar calificaciones, edades, precios o resultados de cálculos.

20.5 Ejemplo de código

Forma tradicional

```
int a = 15;
```

```
int b = 20;
```

```
int mayor;
```

```
if (a > b) {
```

```
        mayor = a;
    } else {
        mayor = b;
    }
```

```
System.out.println(mayor);
```

Forma abreviada con Math.max()

```
int a = 15;
```

```
int b = 20;
```

```
int mayor = Math.max(a, b);
```

```
System.out.println(mayor);
```

Forma abreviada con Math.min()

```
int a = 15;
```

```
int b = 20;
```

```
int menor = Math.min(a, b);
```

```
System.out.println(menor);
```

20.6 Comparación con una implementación tradicional

Implementación tradicional

Utiliza una estructura if-else para comparar valores.

Requiere más líneas de código.

Es más extensa y repetitiva.

Math.max() / Math.min()

Realiza la comparación con una sola instrucción.

Reduce la cantidad de código.

Hace el código más limpio y legible.

Implementación tradicional

La comparación la realiza el programador.

Math.max() / Math.min()

Java realiza la comparación automáticamente.

Referencias bibliográficas

1. Oracle. (s. f.). *Java Documentation*. Recuperado de <https://docs.oracle.com/en/java/>
2. Oracle. (s. f.). *The Java™ Tutorials*. Recuperado de <https://docs.oracle.com/javase/tutorial/>
3. Oracle. (s. f.). *Java Platform, Standard Edition API Specification*. Recuperado de <https://docs.oracle.com/en/java/javase/>
4. Baeldung. (s. f.). *Java Tutorials*. Recuperado de <https://www.baeldung.com/java-tutorial>
5. GeeksforGeeks. (s. f.). *Java Programming Language*. Recuperado de <https://www.geeksforgeeks.org/java/>
6. OpenJDK. (s. f.). *OpenJDK Documentation*. Recuperado de <https://openjdk.org/>
7. Bloch, J. (2018). *Effective Java* (3.^a ed.). Addison-Wesley Professional.
8. Sierra, K., & Bates, B. (2022). *Head First Java* (3.^a ed.). O'Reilly Media.